DIMENSIONAL REDUCTION ON STEROIDS: GEOMETRIC CHARACTERIZATION OF FEASIBLE PARAMETER SPACES IN BUCKINGHAM PI

VASILII PUSTOVOIT , UTKARSH MALI , AND [CO-AUTHORS?]

Abstract. [Check last] We study the inverse dimensional reduction problem arising in Buckingham Pi analysis: given a sampled point $\mathbf{d}$ in dimensionless space $\mathbb{R}^m$ and a map f encoding the Buckingham Pi exponents (linear: $A\mathbf{p} = \mathbf{d}$, or non-linear monotonic), recover a physical parameter vector $\mathbf{p} \in \mathbb{R}^n$ satisfying the map and the physical constraints $\Lambda\mathbf{p} \leq \mathbf{c}$. We present STEROID (Simplicial TEssellation Reduction Of Invariant Domains), which preprocesses the feasible polytope once and answers subsequent queries in $O(1)$ time. We benchmark STEROID against six alternative methods across a grid of problem sizes (n, m). For *point queries alone*, compiled Dykstra is faster at $m \geq 4$ with zero preprocessing, with a crossover at $m \approx 3$–4: the two methods scale in opposite directions with m, and no preprocessing amortization justifies STEROID for pure feasibility queries. However, STEROID provides precomputed smooth $\partial\mathbf{p}/\partial\mathbf{d}$, exact one-pass uniform sampling of the feasible dimensionless region, non-linear monotonic extensions, and geometric characterization of the feasible dimensionless region with no per-query analogue.

1. Introduction.

Buckingham's Π-theorem [7] states that any physically meaningful equation involving n dimensional quantities can be expressed in terms of $m = n - k$ independent dimensionless groups, where k is the rank of the dimensional matrix (i.e. number of dimensions in kernel of the problem). Dimensional analysis has become a core tool in fluid mechanics, astrophysics, materials science, and engineering design [3, 16, 23, 21]: working in dimensionless space reduces experimental parameter sweeps from n-dimensional to m-dimensional.

Applications increasingly require the *inverse* direction: given a query point $\mathbf{d}$ sampled in dimensionless space (e.g., a grid point in a design sweep), find one or more physical parameter vectors $\mathbf{p}$ that reproduce $\mathbf{d}$ under the Buckingham map and satisfy all physical constraints (ranges of validity, material limits, operating bounds). This problem is under-determined—the preimage $A^{-1}\mathbf{d}$ is an affine subspace of dimension $n-m$—and the constraint set restricts it to a (possibly empty) convex polytope within that affine subspace.

To our knowledge, no dedicated open-source solver exists for this problem. Gorard et al. [12] identify the lack of guarantees that dimensionally reduced parameter combinations remain physically valid as an open challenge in nuclear fusion and high-energy astrophysics. Practitioners typically either resort to brute-force rejection sampling (prohibitively slow), or run a linear program (LP) per query without exploiting the repeated structure of the parameter sweep [3, 16].

In this paper we benchmark existing methods for solving the inverse dimensional reduction problem and introduce a novel approach, STEROID, as an alternative. The closest algorithmic relative is explicit linear Model Predictive Control (MPC) [4], which precomputes a piecewise-affine (PWA) map from parameter to optimizer by partitioning the parameter space into critical regions via Karush-Kuhn-Tucker (KKT) conditions. The resulting lookup table supports $O(\log R)$ query cost (R = number of regions) via binary search trees [28]. The Delaunay Triangulation approach (a big part of STEROID method) was also explored for explicit MPC by [25].

The paper is organized as follows. Section 2 formalizes the inverse problem. Section 3 describes each method, including the full STEROID derivation and correctness proof. Sections 5–7 characterize STEROID's unique geometric capabilities, non-linear extension, and GPU-friendly point location. Section 8 describes the experimental setup, Section 9 presents benchmark results, Section 10 demonstrates applications,

and Section 11 provides a method selection guide and discusses limitations.

2. Problem Statement. [Not checked yet]

We work entirely in log-space. Physical variables $\mathbf{p} \in \mathbb{R}^n$ are the logs of positive dimensional quantities; dimensionless variables $\mathbf{d} \in \mathbb{R}^m$ are the logs of Buckingham groups; all constraints are linear in these log variables.

The physical parameter space is $\mathbb{R}^n$. Physical constraints take the form

$$\Lambda \mathbf{p} \leq \mathbf{c}, \tag{2.1}$$

where $\Lambda \in \mathbb{R}^{K \times n}$ and $\mathbf{c} \in \mathbb{R}^K$. We assume the feasible set $\mathcal{P} = \{\mathbf{p} : \Lambda \mathbf{p} \leq \mathbf{c}\}$ is a compact convex polytope (i.e., full-dimensional and bounded).

By the Buckingham Π-theorem, there exists a matrix $A \in \mathbb{R}^{m \times n}$ with $\operatorname{rank}(A) = m < n$ such that the dimensionless variables are

$$A\mathbf{p} = \mathbf{d}. \tag{2.2}$$

The feasible dimensionless region is $\mathcal{D}^* = \{A\mathbf{p} : \mathbf{p} \in \mathcal{P}\}$, the projection of $\mathcal{P}$ under A. This is itself a compact convex polytope in $\mathbb{R}^m$.

Given a query $\mathbf{d} \in \mathbb{R}^m$, an algorithm that solves the inverse problem should find $\mathbf{p} \in \mathbb{R}^n$ satisfying (2.1) and (2.2), or report infeasibility, i.e. return $\emptyset$ if $\mathbf{d} \notin \mathcal{D}^*$.

In practice, queries arrive as a batch $G = \{\mathbf{d}_1, \ldots, \mathbf{d}_Q\}$ (e.g., a regular N^m-point Cartesian grid in $[-1, 1]^m$ in one example), and the problem parameters $A, \Lambda, \mathbf{c}$ are fixed across all queries.

3. Methods.

All methods in this paper exploit the following algebraic reduction.

PROPOSITION 3.1. *The general solution to* $A\mathbf{p} = \mathbf{d}$ *is*

$$\mathbf{p} = A^\dagger \mathbf{d} + N\mathbf{z}, \qquad \mathbf{z} \in \mathbb{R}^{n-m}, \tag{3.1}$$

where $A^\dagger \in \mathbb{R}^{n \times m}$ *is the Moore-Penrose pseudoinverse and* $N \in \mathbb{R}^{n \times (n-m)}$ *is a basis for* $\ker(A)$. *Substituting into* (2.1) *reduces the problem to an LP in* $\mathbb{R}^{n-m}$*:*

$$(\Lambda N)\mathbf{z} \leq \mathbf{c} - \Lambda A^\dagger \mathbf{d}. \tag{3.2}$$

The key observation is that (3.2) lives in $\mathbb{R}^{n-m}$, not $\mathbb{R}^n$. As m increases (holding n fixed), the LP dimension $n - m$ *decreases*. Any per-query method exploits this automatically.

3.1. LP Simplex and Interior Point (HiGHS). [Not checked yet]

The most direct approach: solve (3.2) as an LP feasibility problem with HiGHS [15]. Simplex is exponential in the worst case but typically performs $O((n-m)K)$ pivot steps in practice; interior point achieves polynomial worst-case $O((n-m)^{3.5} \log(1/\varepsilon))$. No preprocessing beyond computing N, $A^\dagger$ once ($O(n^2 m)$ via SVD). **[Sources for these numbers? did not find them in the HiGHS 2018 paper]**

3.2. Chebyshev Center LP. [Not checked yet]

Find the Chebyshev center of (3.2): the largest inscribed ball maximizes minimum slack, giving the most "interior" feasible point. Solved as an LP with one additional radius variable [5]. **[Do not have access to this manuscript. Find another one]** Useful when downstream applications require $\mathbf{p}$ to be far from constraint boundaries.

3.3. Dykstra's Alternating Projection. [Not checked yet]

Dykstra's algorithm [10] finds the projection onto the intersection of convex sets by cyclic orthogonal projection with incremental corrections. Applied to (3.2): project $\mathbf{z}$ onto each half-space $(\Lambda N)_k \mathbf{z} \leq (\mathbf{c} - \Lambda A^\dagger \mathbf{d})_k$ in round-robin order. Projection onto one half-space costs $O(n-m)$. Per-query scaling is then $O(C \cdot K \cdot (n-m))$, $C \approx 5$–20 cycles in practice, which decreases as m increases.

Dykstra terminates when the maximum constraint violation falls below a tolerance ε. The returned point satisfies $(\Lambda N)_k \mathbf{z} \leq (\mathbf{c} - \Lambda A^\dagger \mathbf{d})_k + \varepsilon$ for all k, so feasibility holds up to ε.

3.4. ADMM and Kaczmarz. [Not checked yet]

Alternating Direction Method of Multipliers (ADMM) [6] solves the feasibility problem via consensus splitting, with similar per-iteration cost to Dykstra but easier parameter tuning for ill-conditioned constraint matrices. ADMM converges in $O(1/\varepsilon)$ iterations for the feasibility residual. Randomized Kaczmarz [26] uses random constraint selection and converges exponentially in expectation: $\mathbb{E}\|\mathbf{z}_t - \mathbf{z}^*\|^2 \leq (1 - \kappa^{-2})^t \|\mathbf{z}_0 - \mathbf{z}^*\|^2$, where κ is the scaled condition number of ΛN. A gradient-penalty method adds a differentiable penalty $\rho \| \max(0, (\Lambda N)\mathbf{z} - (\mathbf{c} - \Lambda A^\dagger \mathbf{d}))\|^2$ to a quadratic objective and minimizes via gradient descent, increasing ρ progressively until feasibility is achieved.

3.5. STEROID: Vertex Enumeration + Delaunay Triangulation. [Not checked yet]

STEROID invests in a one-time preprocessing step: it identifies the feasible region, projects it into dimensionless space, triangulates it, and precomputes an affine map for each simplex. Subsequent queries reduce to locating the containing simplex and evaluating a single matrix-vector product.

We now walk through the construction step by step.

3.5.1. Identifying the feasible vertices. [Not checked yet]

The feasible polytope $\mathcal{P} = \{\mathbf{p} : \Lambda \mathbf{p} \leq \mathbf{c}\}$ is defined by K linear constraints (its H-representation). STEROID first converts $\mathcal{P}$ to its V-representation—a list of vertices $W = \{\mathbf{w}_1, \ldots, \mathbf{w}_V\}$—using the Parma Polyhedra Library [2]. By McMullen's Upper Bound Theorem [18], which bounds the number of faces of a polytope with K facets in $\mathbb{R}^n$, the vertex count satisfies $V \leq \binom{K}{\lfloor n/2 \rfloor}$.

3.5.2. Projection into dimensionless space. [Not checked yet]

Each vertex $\mathbf{w}_i \in \mathbb{R}^n$ is mapped to dimensionless space via $\mathbf{v}_i = A\mathbf{w}_i \in \mathbb{R}^m$. The projected vertices $\{\mathbf{v}_1, \ldots, \mathbf{v}_V\}$ span the feasible dimensionless region $\mathcal{D}^*$.

Degenerate projections. Multiple physical vertices may project to the same (or nearly the same) dimensionless point when the polytope $\mathcal{P}$ has edges nearly aligned with $\ker(A)$. In this case the Delaunay triangulation produces near-degenerate simplices ($\det P_i \approx 0$), and the deprojection matrix $E_i = P_i' P_i^{-1}$ becomes ill-conditioned. A natural remedy, left for future implementation, is to merge projected vertices within a relative tolerance $\varepsilon_{\text{merge}}$ before triangulation, retaining among merged candidates the parent vertex farthest from all constraint boundaries (maximizing slack).

3.5.3. Triangulating the dimensionless region. [Not checked yet]

We compute the Delaunay triangulation of the projected vertices using CGAL [27], producing a set of simplices $\{T_1, \ldots, T_S\}$ that partition $\mathcal{D}^*$. In the worst case, $S = O(V^{\lceil m/2 \rceil})$ [18].

3.5.4. Obtaining physical values from a query point. [Not checked yet] Now take an arbitrary simplex T_i and a query point $\mathbf{d}$ inside it. Each simplex has $m+1$ vertices in dimensionless space; we pick one as the "base" vertex $\mathbf{v}_{i,0}$ and form the difference vectors from it. Any point $\mathbf{d}$ inside T_i can then be written as:

$$\mathbf{d} = \mathbf{v}_{i,0} + \sum_{j=1}^{m} \mu_j (\mathbf{v}_{i,j} - \mathbf{v}_{i,0}). \tag{3.3}$$

Where we solve for $\vec{\mu} = \{\mu_i\}$. If no simplex contains $\mathbf{d}$, then $\mathbf{d} \notin \mathcal{D}^*$ and we report infeasibility.

To get the physical parameters, we remember that each projected vertex $\mathbf{v}_{i,j}$ has a "parent" vertex $\mathbf{w}_{i,j}$ in physical space (the vertex that was projected to produce it: $A\mathbf{w}_{i,j} = \mathbf{v}_{i,j}$). We apply the *same* barycentric coordinates μ_j to the parent vertices:

$$\mathbf{p} = \mathbf{w}_{i,0} + \sum_{j=1}^{m} \mu_j (\mathbf{w}_{i,j} - \mathbf{w}_{i,0}). \tag{3.4}$$

3.5.5. Precomputing the deprojection map. [Not checked yet] Since the difference vectors in (3.3) and (3.4) depend only on the simplex (not on the query point), we can precompute everything into a compact form.

Define $P_i \in \mathbb{R}^{m\times m}$ as the matrix whose columns are the difference vectors $\mathbf{v}_{i,j} - \mathbf{v}_{i,0}$ for $j = 1, \ldots, m$, and $P_i' \in \mathbb{R}^{n\times m}$ as the matrix whose columns are the corresponding physical differences $\mathbf{w}_{i,j} - \mathbf{w}_{i,0}$.

With this notation, (3.3) becomes $\mathbf{d} = \mathbf{v}_{i,0} + P_i\boldsymbol{\mu}$. Since T_i is a non-degenerate simplex, $\det P_i \neq 0$, and we can solve for the barycentric coordinates:

$$\boldsymbol{\mu} = P_i^{-1}(\mathbf{d} - \mathbf{v}_{i,0}). \tag{3.5}$$

Substituting into (3.4):

$$\mathbf{p} = \mathbf{w}_{i,0} + P_i' P_i^{-1}(\mathbf{d} - \mathbf{v}_{i,0}) = \mathbf{w}_{i,0} - P_i' P_i^{-1}\mathbf{v}_{i,0} + P_i' P_i^{-1}\mathbf{d}. \tag{3.6}$$

The first two terms give a vector that depends only on the simplex. The last term is a matrix (also depending only on the simplex) multiplied by the query point. We can therefore write:

$$\mathbf{p} = \mathbf{e}_i + E_i\,\mathbf{d}, \qquad E_i = P_i' P_i^{-1}, \qquad \mathbf{e}_i = \mathbf{w}_{i,0} - E_i\,\mathbf{v}_{i,0}. \tag{3.7}$$

Both $\mathbf{e}_i \in \mathbb{R}^n$ and $E_i \in \mathbb{R}^{n\times m}$ are computed once per simplex during preprocessing and stored for retrieval.

3.5.6. Query evaluation. [Not checked yet] At query time, STEROID performs two operations:

1. *Point location*: find the simplex T_i containing $\mathbf{d}$ via CGAL's triangulation walk—starting from a previously located simplex and following facet adjacencies until $\mathbf{d}$ is enclosed.
2. *Affine map*: compute $\mathbf{p} = \mathbf{e}_i + E_i\mathbf{d}$ (one matrix-vector multiply, $O(nm)$ operations).

If the walk exits the convex hull of $\mathcal{D}^*$, the query is infeasible. The deprojection data $(\mathbf{e}_i, E_i)$ can also be evaluated in batch: a matrix of query points yields a matrix of physical parameters, making the linear algebra embarrassingly parallel.

3.5.7. Correctness. **[Not checked yet]** The key property of this construction is that it always produces a feasible physical point.

THEOREM 3.2. *For any* $\mathbf{d}$ *interior to a simplex* $T_i \subset \mathcal{D}^*$*, the deprojected point* $\mathbf{p} = \mathbf{e}_i + E_i\mathbf{d}$ *satisfies: (i)* $A\mathbf{p} = \mathbf{d}$*, and (ii)* $\Lambda\mathbf{p} \leq \mathbf{c}$.

Proof. Since $\mathbf{d}$ lies inside T_i, the barycentric coordinates $\mu_j \geq 0$ with $\sum_j \mu_j \leq 1$. Setting $\mu_0 = 1 - \sum_{j=1}^{m} \mu_j$, the deprojected point is $\mathbf{p} = \sum_{j=0}^{m} \mu_j \mathbf{w}_{i,j}$—a convex combination of the physical vertices. Since $A\mathbf{w}_{i,j} = \mathbf{v}_{i,j}$ by construction: $A\mathbf{p} = \sum_j \mu_j \mathbf{v}_{i,j} = \mathbf{d}$. Since each $\mathbf{w}_{i,j} \in \mathcal{P}$ and $\mathcal{P}$ is convex, $\mathbf{p} \in \mathcal{P}$, i.e., $\Lambda\mathbf{p} \leq \mathbf{c}$. ∎

Remark 3.3. The deprojection map $\mathbf{d} \mapsto \mathbf{p}$ is piecewise affine and generally *discontinuous* across simplex boundaries: adjacent simplices T_i and T_j sharing a facet may have different parent vertices, producing different $\mathbf{p}$ values at the shared boundary. Feasibility is preserved (both outputs lie in $\mathcal{P}$ and satisfy $A\mathbf{p} = \mathbf{d}$), but the map is multi-valued at boundaries. The C^k spline interpolation of Section 5.4 resolves this by enforcing smoothness across simplex boundaries.

3.5.8. Scaling. **[Not checked yet]** The per-query cost has two components: (i) the triangulation walk, costing $O(S^{1/m})$ simplex traversals in expectation (each requiring $O(m)$ facet tests), giving $O(m \cdot S^{1/m})$; and (ii) the matrix multiply $E_i\mathbf{d}$, costing $O(nm)$. Both grow with m, making STEROID *slower* as m increases—the opposite scaling from Dykstra.

Preprocessing is dominated by the Delaunay triangulation, with worst-case $O(V^{\lceil m/2 \rceil})$ simplices. This becomes impractical for $m \geq 6$.

4. Complexity Summary. **[Not checked yet]**

TABLE 1
Asymptotic complexity. K: constraints; n, m: dimensions; $V \sim K^{\lfloor n/2 \rfloor}$: vertices; C: convergence cycles. †Typical-case; simplex is exponential in the worst case.

Method	Preprocessing	Per query	Memory	Trend in m
LP simplex	$O(1)$	$O((n-m)K)^{\dagger}$	$O(nK)$	Gets faster
LP IPM	$O(1)$	$O((n-m)^{3.5})$	$O(nK)$	Gets faster
Chebyshev	$O(1)$	$O((n-m)K)^{\dagger}$	$O(nK)$	Gets faster
Dykstra	$O(1)$	$O(CK(n-m))$	$O(nK)$	Gets faster
ADMM	$O(1)$	$O(CK(n-m))$	$O(nK)$	Gets faster
Kaczmarz	$O(1)$	$O(C(n-m))$	$O(nK)$	Gets faster
STEROID	$O(V^{\lceil m/2 \rceil})$	$O(m \cdot S^{1/m} + nm)$	$O(V^{\lceil m/2 \rceil} n)$	Gets slower

5. Geometric Capabilities of STEROID. **[Not checked yet]**

Beyond point queries, STEROID's triangulation enables tasks inaccessible to per-query methods.

5.1. Feasible volume computation. **[Not checked yet]**

$$\text{vol}(\mathcal{D}^*) = \sum_i \frac{|\det P_i|}{m!}. \tag{5.1}$$

Exact, $O(Sm^3)$ (one determinant per simplex). Monte Carlo alternatives converge as $O(1/\sqrt{N})$. Application: uncertainty quantification—"what fraction of dimensionless design space is physically realizable?"

5.2. Exact uniform sampling. [Not checked yet] To sample uniformly in dimensionless space $\mathcal{D}^*$: select simplex T_i with probability $\propto |\det P_i|/m!$ (proportional to its volume); sample $\mathbf{d} \sim \text{Uniform}(T_i)$ via Dirichlet$(1, \ldots, 1)$ barycentric coordinates; recover $\mathbf{p} = \mathbf{e}_i + E_i\mathbf{d}$.

To sample uniformly in *physical* space $\mathcal{P}$, the simplex selection probabilities must be adjusted by the Jacobian of the deprojection map: select T_i with probability $\propto |\det P_i| \cdot |\det(E_i^\top E_i)|^{1/2}/m!$, which accounts for the volume distortion of the affine map E_i from $\mathbb{R}^m$ to $\mathbb{R}^n$. Both variants are exact one-pass samplers. The Volesti library [8] provides hit-and-run MCMC sampling within polytopes, but requires a burn-in period of $O(n^3)$ steps before samples are approximately uniform, and the approximation error is controlled only in distribution. STEROID's sampler produces exactly uniform samples with no burn-in and no tolerance parameter.

5.3. Phase diagrams of active constraint sets. [Not checked yet] Each physical constraint $(\Lambda\mathbf{p})_k \leq c_k$ is either *active* (tight, $= c_k$) or *inactive* (strict, $< c_k$) at a given point $\mathbf{p}$. The set of active constraints changes as $\mathbf{d}$ moves through $\mathcal{D}^*$: in one region of dimensionless space the amplitude bound may be the binding constraint, while in another it may be the temperature bound. Knowing which constraint is active at each operating point is directly useful for engineering: it tells the designer which physical limit is the bottleneck for each target dimensionless state.

STEROID makes this information available without any additional computation. For each simplex T_i, the parent vertices $\{\mathbf{w}_{i,0}, \ldots, \mathbf{w}_{i,m}\}$ already lie on the boundary of $\mathcal{P}$, so their active constraint sets $\mathcal{A}_{i,j} = \{k : (\Lambda\mathbf{w}_{i,j})_k = c_k\}$ are known from the vertex enumeration step. We define the dominant active set of simplex T_i as the union $\mathcal{A}_i = \bigcup_j \mathcal{A}_{i,j}$. Coloring each simplex in the triangulation by $\mathcal{A}_i$ produces a map in $\mathcal{D}^*$ showing which physical constraints govern each region of dimensionless space—a phase diagram in the thermodynamic sense.

In explicit MPC [28], a similar map is produced via KKT critical regions, where each region corresponds to exactly one active set by construction. STEROID's Delaunay simplices are determined by geometry rather than optimality, so a single simplex may nominally span two active sets near a transition; the phase diagram is therefore an approximation near boundaries between phases, improving as the triangulation is refined.

5.4. Smooth $d \to p$ manifolds via spline interpolation. [Not checked yet]

The Delaunay triangulation provides a simplicial mesh on $\mathcal{D}^*$ suitable for C^k spline interpolation [17]. Replacing the piecewise-affine STEROID map with a C^k spline yields:

- Analytical sensitivity: $\partial\mathbf{p}/\partial\mathbf{d}$ available everywhere in $\mathcal{D}^*$.
- Smooth optimization on the feasible manifold.
- Geodesic computation with well-defined curvature.

The analytic center [20] achieves C^∞ but requires online optimization per query. STEROID + splines achieves C^k with precomputed $O(1)$ evaluation. Constructing splines that are continuous across simplex boundaries (i.e., achieving $k \geq 1$) requires choosing the spline coefficients to satisfy inter-element compatibility conditions; this is a standard problem in finite element analysis [17, 9] and is deferred to future work.

6. Extension to Non-Linear Monotonic Maps. [Not checked yet]

DEFINITION 6.1. *$f : \mathbb{R}^n \to \mathbb{R}^m$ is* coordinate-monotonic *if each f_j is monotonically increasing or decreasing in each p_i separately.*

Physical examples include Arrhenius reaction rate laws ($k \propto e^{-E_a/RT}$, monotone

in temperature T), Cobb-Douglas production functions ($Y = AK^\alpha L^\beta$, monotone in capital K and labor L), and power-law rheology models where viscosity depends monotonically on shear rate and temperature [23, 22].

PROPOSITION 6.2. *If f is coordinate-monotonic and $\mathcal{P}$ is a compact convex polytope, then the vertices of $\mathcal{P}$ map to extreme points of $f(\mathcal{P})$.*

Note that $f(\mathcal{P})$ is *not* necessarily convex for general coordinate-monotonic f. However, the STEROID pipeline does not require convexity of $f(\mathcal{P})$: it only requires that the Delaunay triangulation of the projected vertices covers the region of interest. In practice, we triangulate the convex hull of $\{f(\mathbf{w}_i)\}$, which is a convex superset of $f(\mathcal{P})$. Query points that fall inside the convex hull but outside $f(\mathcal{P})$ will still receive a deprojected $\mathbf{p}$ that satisfies $\mathbf{p} \in \text{conv}(\mathcal{P}) = \mathcal{P}$ (by convexity of $\mathcal{P}$), but with $f(\mathbf{p}) \neq \mathbf{d}$ in general—the deprojected point is feasible but the forward map is only approximate. For coordinate-monotonic f that are additionally *componentwise convex* (or concave), $f(\mathcal{P})$ is itself convex and the convex hull equals $f(\mathcal{P})$ exactly.

The STEROID pipeline proceeds as before: enumerate vertices of $\mathcal{P}$, project via f (function evaluation rather than matrix multiply), triangulate. The per-simplex map becomes a piecewise-linear approximation to f^{-1}.

Approximation error. Since f is under-determined ($m < n$), f^{-1} is not a function in the classical sense. However, STEROID's piecewise-linear construction implicitly selects a *section* $\sigma : \mathcal{D}^* \to \mathcal{P}$ (a right inverse of f) defined by the barycentric interpolation of parent vertices. Let σ^* denote the "true" section that would be obtained by exact inversion (e.g., the section passing through the same parent vertices with $f(\sigma^*(\mathbf{d})) = \mathbf{d}$ exactly). On each simplex T_i with diameter h_i, σ agrees with σ^* at the vertices, so by standard multivariate Lagrange interpolation error [9]:

$$\|\sigma(\mathbf{d}) - \sigma^*(\mathbf{d})\| = O\big(h_i^2 \|D^2\sigma^*\|_\infty\big)\,. \tag{6.1}$$

Finer triangulation (more vertices or adaptive refinement) reduces h_i and improves accuracy, but also increases precompute time.

Uniqueness. Explicit MPC requires $f = A$ (linear). Analytic center and Dykstra require convex $f(\mathcal{P})$ (non-linear f may destroy convexity). Differentiable convex optimization layers [1] embed a convex program as a layer in a neural network, enabling gradient-based learning through the solution map; but this still requires $f(\mathcal{P})$ to be convex at each forward pass. Only STEROID operates directly on $\mathcal{P}$, avoiding any convexity requirement on f.

7. GPU-Friendly Point Location (Projected Performance). [Not checked yet]

The current STEROID bottleneck for large batches is CGAL's triangulation walk (pointer-chasing, cache-unfriendly on GPU). The matrix multiply $E_i\mathbf{d}$ is already perfectly SIMD-parallel.

7.1. Spatial hashing. [Not checked yet]

During preprocessing, insert each simplex into a hash table keyed by the Morton code (Z-order curve) [19] of its bounding grid cell. Per-query: one hash lookup $O(1)$ + $O(m)$ barycentric containment check.

7.2. BVH for higher dimensions. [Not checked yet]

Morton encoding requires > 64 bits for $m \geq 9$. A Bounding Volume Hierarchy (BVH) provides $O(m \log V)$ point location. NVIDIA RTX cores include dedicated BVH traversal hardware applicable to simplex point location.

TABLE 2
Estimated per-query FLOPs with spatial-hash point location ($C = 10$ for Dykstra). ***[Verify with hardware benchmarks.]***

m	n	STEROID (hash+matmul)	Dykstra	STEROID speedup
2	6	~ 12	~ 960	$\sim 80\times$
3	6	~ 18	~ 660	$\sim 37\times$
5	6	~ 30	~ 330	$\sim 11\times$
5	10	~ 50	~ 750	$\sim 15\times$
8	12	~ 96	~ 1080	$\sim 11\times$

7.3. Warp-divergence advantage. **[Not checked yet]** Dykstra's variable iteration count causes GPU warp divergence: threads in a warp must wait for the slowest query before proceeding, wasting compute on well-conditioned queries. STEROID's fixed-cost pipeline (one lookup + one matmul) eliminates warp divergence entirely, providing a fundamental architectural advantage for large batches on GPU. Empirical quantification via CUDA profiling is deferred to future work.

8. Experimental Setup. [Not checked yet]

8.1. Problem generation. **[Not checked yet]** We generate synthetic problems on an (n, m) grid with $n \in \{4, 6, 8, 10, 12, 15, 20\}$ and $m \in \{2, 3, 4, 5, 6, 8\}$ (valid pairs: $m < n$). For each pair:

1. $A \in \mathbb{R}^{m \times n}$: random normal, orthogonalized to $\text{rank}(A) = m$.
2. $K = 3(n + m)$ constraints: $\Lambda \in \mathbb{R}^{K \times n}$ with entries in $[-1, 1]$; $\mathbf{c}$ shifted to ensure feasibility.
3. Query set G: 11^m-point Cartesian grid in $[-1, 1]^m$, capped at 500.

8.2. Implementation. **[Not checked yet]** Python methods (LP simplex, LP IPM, Chebyshev) use HiGHS via the `highspy` Python bindings with default solver settings. Dykstra, ADMM, Kaczmarz, gradient penalty, and LP-HiGHS are implemented as standalone C++ binaries compiled with `-O3 -march=native` and linked against HiGHS [**HiGHS2022**]. STEROID is compiled with the same flags, using CGAL [27] for triangulation and PPL [2] for vertex enumeration. For each method, preprocessing (matrix factorizations, triangulation) is timed separately from per-query evaluation. Python method timings reflect interpreter overhead; C++ timings are hardware-limited.

8.3. Timing. **[Not checked yet]** Per method, we record: preprocessing time t_{pre} (s), mean per-query time t_{q} (ms), p_{99} query time, and feasibility rate. $N_{\text{rep}} = 5$ repetitions; median reported. **Hardware: [CPU model, RAM, OS, compiler version.]**

9. Results. [Not checked yet]

9.1. Per-query speed. **[Not checked yet]** Figure 1 shows mean query time across the (n, m) grid.

C++ Dykstra achieves 0.001–0.013 ms per query across all tested pairs with zero preprocessing. STEROID is $\sim$7$\times$ faster at $m = 2$ and $\sim$10$\times$ slower at $m = 5$. Crossover at $m \approx 3$–4, consistent with opposite scaling predicted in Section 3. Python LP methods are $\sim$100–3000$\times$ slower than Dykstra; C++ LP-HiGHS is $\sim$30–100$\times$ slower.

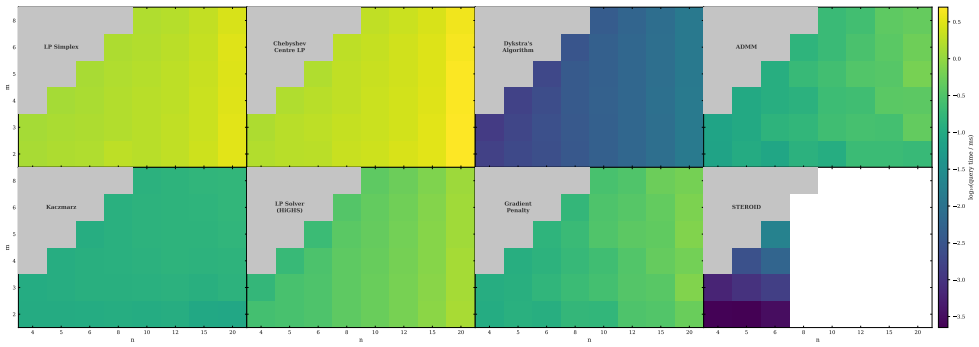

FIG. 1. *Mean query time (ms, log scale) vs. (n, m) per method. White: missing data (timeout or infeasible preprocessing).*

9.2. Preprocessing. [Not checked yet] Figure 2 shows preprocessing time.

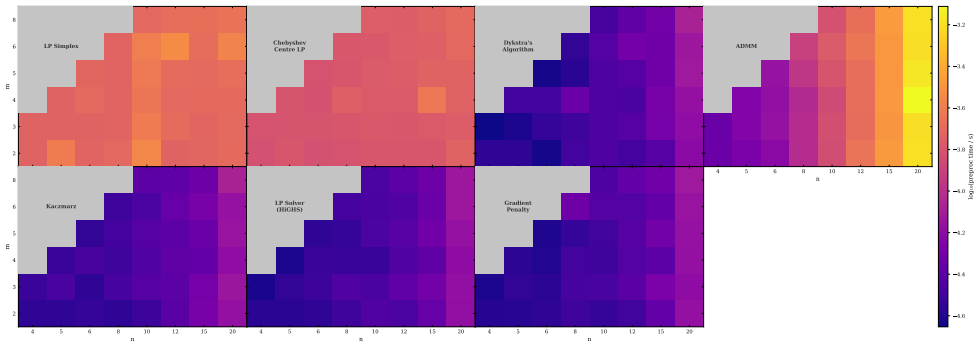

FIG. 2. *Preprocessing time (s, log scale) vs. (n, m). White: exceeded timeout or not tested.*

STEROID preprocessing grows exponentially in m and becomes infeasible at $m \geq 6$ in our tests (the largest tested STEROID instance, $n = 6$, $m = 5$, requires ∼0.09 s). All per-query methods have ≤ 1 ms preprocessing (SVD of A only).

9.3. Break-even query count. [Not checked yet] Figure 3 shows $Q^* = t_{\text{pre}}^{\text{STEROID}} / (t_{\text{q}}^{\text{method}} - t_{\text{q}}^{\text{STEROID}})$ for (n, m) pairs where STEROID is faster per query $(m \leq 3)$.

At $m = 2$, STEROID preprocessing takes 0.0002–0.0008 s (depending on n), while the per-query advantage over Dykstra is ∼0.002 ms. This gives $Q^* \approx 100$–400 queries to break even—comparable to the standard $11^2 = 121$ grid. At $m = 3$, preprocessing takes 0.0003–0.0016 s with a per-query advantage of ∼0.001 ms, giving $Q^* \approx 300$–1600. The standard $11^3 = 1331$ grid approaches break-even at the larger (n, m) pairs.

9.4. Speedup vs. STEROID. [Not checked yet] Figure 4 shows per-query speedup of each method relative to STEROID.

Dykstra is faster than STEROID for all tested pairs with $m \geq 4$.

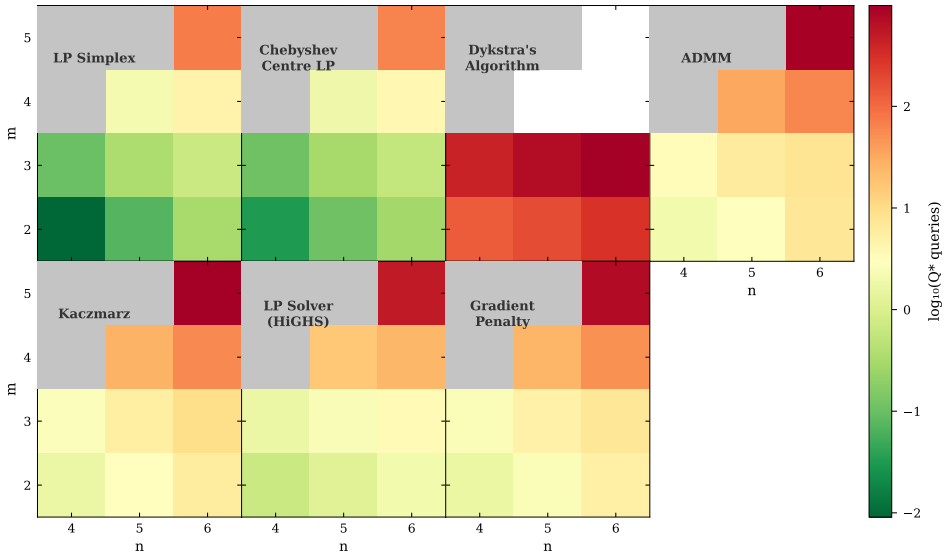

FIG. 3. *Break-even query count* $\log_{10}(Q^*)$ *for* (n, m) *pairs where STEROID has per-query advantage. Red = many queries to amortize; blue = few.*

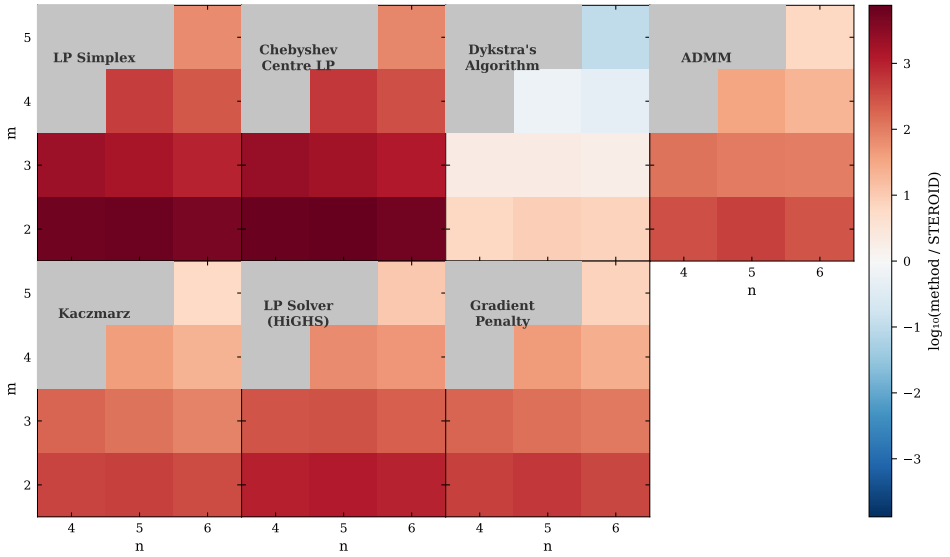

FIG. 4. *Query speedup vs. STEROID (log ratio; blue = faster than STEROID).*

10. Applications. [Not checked yet]

We now demonstrate STEROID's unique capabilities on two concrete applications drawn from the literature, selected because they require capabilities that per-query methods genuinely cannot provide. We also discuss two motivating examples where STEROID addresses an open gap identified in the literature, even though full numerical demonstrations are deferred.

10.1. Ultrasonic micro-injection molding: inverse design and sensitivity. [Not checked yet]

Salazar-Meza et al. [23] apply Buckingham Pi analysis to ultrasonic micro-injection molding (UMIM), fitting power-law models for energy consumption Q and final Young's modulus E of the molded part. With $n = 7$ physical variables (amplitude A, action time U_t, plunger power P, flexural modulus S_f, mold temperature T_M, thermal conductivity λ, output property) and $m = 3$ dimensionless groups ($\Pi_0 = Q/(U_t P)$, $\Pi_1 = A^4 S_f T_M \lambda/(U_t P^2)$, $\Pi_2 = A^3 E/(U_t P)$), the fitted models read in log-space:

$$\log E = \log(5.82{\times}10^{-8}) - 1.03\log A + 0.508\log U_t + 0.016\log P + 0.492\log(S_f T_M \lambda),$$
$$\log Q = \log(9.05{\times}10^{5}) + 2.20\log A + 0.449\log U_t - 0.101\log P + 0.551\log(S_f T_M \lambda).$$

The exponent matrix A is read directly from these equations. Process constraints are log-linear box bounds: $A \in [22.5, 56.2]$ μm, $U_t \in [4, 5]$ s, IF ≤ 4000 N, $T_M \in [323, 373]$ K, plus the feasibility threshold $\log \Pi_1 > -9 \log 10$ (which is also linear in log-space).

Gap in the existing work. The paper solves only the *forward* direction—given process parameters, predict E and Q. It explicitly leaves the inverse problem unsolved: given a target Young's modulus E^*, which combinations of (A, U_t, P, T_M) achieve it within the feasible operating window? The paper identifies infeasible combinations empirically by discarding trials that fail to produce complete specimens.

What STEROID provides.

1. *Inverse design*: for any target (E^*, Q^*), STEROID returns a feasible process parameter set in $O(1)$ after preprocessing. The feasible region in (Π_1, Π_2) space is a polytope image; its preimage covers all achievable operating points.
2. *Sensitivity surface*: via C^k splines on the triangulation, $\partial E/\partial A$, $\partial E/\partial U_t$, etc. are available as continuous functions over the entire feasible operating space. A process engineer can identify the most sensitive parameter direction and allocate manufacturing tolerance budgets accordingly. No per-query method produces this: LP gives one point at a time with no mesh.
3. *Feasibility characterization*: STEROID computes the exact volume of the achievable (Π_1, Π_2) region and identifies which constraint—amplitude bound, power bound, or temperature bound—is active at each operating point.

[If time permits: implement the UMIM STEROID run numerically and show the sensitivity surface plot as a figure here.]

10.2. Dimensionless robot policy transfer: hardware feasibility. [Not checked yet]

Girard [11] and Hromatko et al. [14] show that control policies trained in dimensionless space transfer zero-shot to any physically similar system. For a pendulum, the dimensionless context is $\mathbf{d} = (q^*, \tau^*_{\max}) = (q/(mgl),\ \tau_{\max}/(mgl))$; for a race car, $m = 5$ dimensionless groups encode mass ratios and scaled traction coefficients. The Buckingham Pi maps are power-law monomials and hence linear in log-space (Theorem 1 of [11] establishes this for the pendulum; the race car uses the same monomial structure but the log-linearity should be verified from the specific group definitions in each application).

Gap identified in the literature. Both papers prove that many physical systems map to the same $\mathbf{d}$, but neither provides an algorithm for finding which hardware configurations are feasible: "*Equation (12) is a mapping from a m dimensional space*

to a $m-d$ dimensional space, and its inverse has multiple solutions" [11] (here d denotes the number of base dimensions in the original text, not our dimensionless vector $\mathbf{d}$). Transferring a trained policy requires identifying real hardware (within actuator datasheets and structural bounds) that achieves the target $\mathbf{d}$.

What STEROID provides. The hardware constraint polytope (motor torque bounds, mass range, link length range from a parts catalogue) is log-linear. STEROID precomputes the image of this polytope in $\mathbf{d}$-space—the set of dimensionless operating points achievable with available hardware—and answers "does hardware configuration $\mathbf{p}$ achieve $\mathbf{d}^*$?" in $O(1)$. For the more practically important question of finding the full family of hardware achieving $\mathbf{d}^*$, STEROID provides exact uniform sampling and the phase diagram of binding constraints (e.g., torque limit vs. mass limit vs. link length limit).

Non-linear extension. Real robot joints have nonlinear inertia and friction terms. If the forward map $\mathbf{d} = f(\mathbf{p})$ is coordinate-monotonic but not log-linear (e.g., nonlinear joint stiffness), LP-per-query becomes non-convex and fails. STEROID's non-linear monotonic extension is the only method that handles this case.

10.3. Open gap: inverse dimensional reduction in fusion reactor design. [Not checked yet]

Gorard et al. [12] identify the problem of ensuring physical validity of dimensionally reduced parameter combinations as an open challenge in nuclear fusion and astrophysics. For tokamak design, the energy confinement scaling laws (IPB98, DS03) are power laws in plasma temperature, density, magnetic field, and geometry, and the Greenwald density limit, Troyon beta limit, and kink stability factor are all log-linear constraints—the full STEROID problem structure.

We note, however, that replacing STEROID here requires care: the expensive step in existing reactor dimensioning studies (e.g., [24]) is a self-consistent nonlinear system solve per design point, not an LP. STEROID addresses the *linear sub-problem* (which parameter combinations satisfy the scaling law and linear constraints), not the full nonlinear reactor code. Its value would be in providing precomputed sensitivity $\partial(R, B, \beta_N)/\partial(\text{exponents})$ for uncertainty propagation through the scaling law, and in generating unbiased samples from the feasible reactor design space for Bayesian inference—neither of which a per-query LP provides.

11. Discussion. [Not checked yet]

11.1. When to use STEROID vs. Dykstra. [Not checked yet]

A critical finding of our benchmark is that STEROID's preprocessing overhead is *not* justified by per-query speed alone: Dykstra is faster at $m \geq 4$ with zero preprocessing. STEROID is the right choice only when the application genuinely requires one of its unique capabilities.

For pure point queries at $m \geq 4$, Dykstra requires no preprocessing and is faster per query than STEROID; there is no benefit to the preprocessing cost. At $m \leq 3$ with very many queries ($Q \gtrsim 10^5$), STEROID's $O(1)$ per-query cost can amortize the preprocessing over the batch. When smooth sensitivity $\partial\mathbf{p}/\partial\mathbf{d}$ is required over all of $\mathcal{D}^*$—for tolerance analysis or gradient-based design optimization—LP and Dykstra are inapplicable: they return isolated points with no mesh structure, and derivatives cannot be formed across queries. Exact uniform sampling of the feasible space is similarly beyond LP or Dykstra; Volesti's MCMC is an approximation with burn-in cost. When the forward map f is coordinate-monotonic but not log-linear, convexity of the feasibility domain is not guaranteed and LP/Dykstra may fail; STEROID

TABLE 3
Method selection guide. "Point query" = find any feasible $\mathbf{p}$ *for a given* $\mathbf{d}$.

Need	Recommended method
Point query only, $m \geq 4$	Dykstra (C++)
Point query only, $m \leq 3$, $Q \gtrsim 10^5$	STEROID
Smooth $\partial\mathbf{p}/\partial\mathbf{d}$ everywhere	STEROID + splines
Exact uniform sampling	STEROID
Non-linear monotonic f	Extended STEROID
Feasible volume or phase diagram	STEROID
$m \geq 6$, any task	Dykstra
Certifiably exact solution	LP (HiGHS)

operates on $\mathcal{P}$ directly and is unaffected. At $m \geq 6$, STEROID's preprocessing grows exponentially and Dykstra becomes the only practical option. When a certifiably exact solution is required, LP via HiGHS [**HiGHS2022**] is preferable to Dykstra (which has a finite termination tolerance); STEROID is exact in the linear case by Theorem 3.2.

Common misconception. It is tempting to argue STEROID is justified whenever preprocessing can be paid once and many queries follow. This is true only at $m \leq 3$ (where STEROID is faster per query) and $Q \gtrsim 10^5$. For $m \geq 4$ and any realistic query count, Dykstra is faster *in total* (preprocessing + all queries). The correct justification for STEROID in those regimes is always one of the unique capabilities in Table 3, not amortized query speed.

11.2. Relationship to explicit MPC. [Not checked yet]

STEROID computes the same mathematical object as explicit MPC [4]: a precomputed piecewise-affine lookup table that maps a low-dimensional parameter $\mathbf{d}$ to a feasible solution $\mathbf{p}$. The connection is genuine: both partition a bounded parameter domain into regions and precompute an affine map per region. However, the two approaches differ in how those regions arise.

In explicit MPC, regions are *critical regions* defined by active constraint sets from the parametric KKT conditions. Their boundaries are determined by the constraints and the objective function, and can be irregular in shape, numerous in count, and expensive to enumerate via multi-parametric programming. In STEROID, regions are *Delaunay simplices* obtained by triangulating the projected feasible vertices. Simplices are well-shaped by the Delaunay circumsphere criterion, and their count depends on the number of vertices rather than the number of constraint inequalities. Scibilia et al. [25] independently proposed using Delaunay tessellation for explicit MPC approximation, noting the same regularity advantage. STEROID can be viewed as an exact variant of that idea applied to the Buckingham Pi setting, where the Delaunay regions coincide with KKT critical regions whenever the feasible polytope is simple (each vertex is defined by exactly n tight constraints). For non-simple polytopes the two decompositions differ: KKT regions may be finer, while Delaunay simplices cover the same domain with fewer, better-conditioned cells.

Buckingham Pi guarantees $m \ll n$, so the effective parameter space is genuinely low-dimensional even for large n. This keeps the Delaunay triangulation tractable at dimensions $m \leq 4$ where explicit MPC would face the curse of dimensionality in KKT enumeration. Additionally, STEROID's non-linear monotonic extension (Section 6)

has no direct counterpart in standard explicit MPC formulations, which assume affine objectives and constraints.

11.3. Limitations and future work. [Not checked yet] STEROID is infeasible at $m \geq 6$ due to exponential growth in simplex count. Generalized barycentric coordinates [13] offer a meshfree alternative: given vertices $\{(\mathbf{v}_i, \mathbf{w}_i)\}$, solve a small LP for barycentric weights at each query and return a convex combination of preimages. This eliminates Delaunay entirely while preserving feasibility guarantees. We plan to benchmark this approach in future work.

The GPU spatial hashing analysis in Section 7 is currently theoretical. Empirical validation on CUDA hardware would confirm the warp-divergence advantage.

The current implementation assumes that the feasible set $\{\mathbf{p} : \Lambda\mathbf{p} \geq \mathbf{c}\}$ is a single convex polytope. Physical problems with disjoint or non-convex feasible regions—arising, for example, from phase coexistence or multi-modal constraints— require extensions. A union-of-polytopes representation can handle such cases by running STEROID independently on each convex component and dispatching at query time based on the simplex lookup; the main cost is that vertex enumeration must be repeated per component.

For problems with very large constraint count K, vertex enumeration via PPL can become the bottleneck. Inexact enumeration strategies—such as pruning low-weight vertices or using randomized double description—can reduce the vertex set at the cost of missing thin feasible slivers. For applications where approximate coverage is acceptable, this offers a practical escape from exponential scaling in K.

Finally, for streaming query settings where the dimensionless point $\mathbf{d}$ arrives incrementally, the full triangulation need not be computed upfront. Lazy triangulation inserts simplices on demand, triggered by the first query near each unexplored region. This amortizes preprocessing over the query stream and can be effective when queries cluster in a small sub-region of the feasible domain.

12. Conclusion. [Not checked yet]

We have benchmarked STEROID against six alternative methods for inverse Buckingham Pi dimensional reduction and critically assessed when its preprocessing overhead is justified. The main conclusions are:

- For *point queries alone*, Dykstra dominates STEROID at $m \geq 4$ with zero preprocessing. The crossover is driven by opposite scaling laws: STEROID's triangulation walk grows with m; Dykstra's null-space projection shrinks.
- STEROID preprocessing becomes exponentially infeasible at $m \geq 6$.
- STEROID's preprocessing is justified only when the application requires at least one of: (i) smooth precomputed $\partial\mathbf{p}/\partial\mathbf{d}$, (ii) exact uniform sampling of the feasible space, or (iii) a non-linear monotonic forward map f for which LP/Dykstra produce non-convex feasibility problems.
- The ultrasonic micro-injection molding application of [23] concretely requires all three: inverse design, tolerance sensitivity, and feasibility characterization. The A matrix is read directly from published dimensional analysis; all constraints are log-linear.
- Spatial hashing replaces CGAL's walk with $O(1)$ lookup and eliminates GPU warp divergence, theoretically restoring STEROID's per-query advantage even at $m \geq 4$—pending empirical validation.

We recommend Dykstra for all pure point-query applications, and STEROID as a *geometric characterization tool* for feasible parameter spaces when sensitivity, sampling, or non-linear maps are required.

Open problems include extending to unions of polytopes (non-convex $\mathcal{P}$), lazy triangulation for online query streams, and empirical GPU validation of the spatial-hashing speedup predicted in Section 7.

Acknowledgements. **[Funding sources, compute resources, collaborators.]**

Appendix A. Benchmark Code. **[Not checked yet]**

All benchmark scripts and results are available at **[repository URL]**. Key scripts: `gen_param_sweep.py` (problem generation), `run_suite.py` (bulk runner), `plot_heatmaps.py` (visualization). A Nix development shell (`flake.nix`) provides a reproducible environment.

Appendix B. STEROID File Formats. **[Not checked yet]**

STEROID stores precomputed deprojection data in a binary format (`.sprj`). The layout (little-endian) is:

Offset	Type	Field
0	`int32`	n (physical dimension)
4	`int32`	m (dimensionless dimension)
8	`uint64`	S (number of simplices)
Repeated S times:		
	`float64[`m`]`	simplex centroid $\bar{\mathbf{v}}_i \in \mathbb{R}^m$
	`float64[`n`]`	translation $\mathbf{e}_i \in \mathbb{R}^n$
	`float64[`$n \times m$`]`	deprojection matrix $E_i \in \mathbb{R}^{n \times m}$ (row-major)

Deprojection at query time: $\mathbf{p} = \mathbf{e}_i + E_i\mathbf{d}$.

References.

[1] Akshay Agrawal et al. "Differentiable Convex Optimization Layers". In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 32. 2019.

[2] Roberto Bagnara, Patricia M. Hill, and Enea Zaffanella. "The Parma Polyhedra Library: Toward a Complete Set of Numerical Abstractions for the Analysis and Verification of Hardware and Software Systems". In: *Science of Computer Programming* 72.1–2 (2008), pp. 3–21. DOI: 10.1016/j.scico.2007.08.001.

[3] Joseph Bakarji et al. "Dimensionally Consistent Learning with Buckingham Pi". In: *Nature Computational Science* 2 (2022), pp. 834–844. DOI: 10.1038/s43588-022-00355-5.

[4] Alberto Bemporad et al. "The Explicit Linear Quadratic Regulator for Constrained Systems". In: *Automatica* 38.1 (2002), pp. 3–20. DOI: 10.1016/S0005-1098(01)00174-1.

[5] Stephen Boyd and Lieven Vandenberghe. *Convex Optimization*. Cambridge University Press, 2004. DOI: 10.1017/CBO9780511804441.

[6] Stephen Boyd et al. "Distributed Optimization and Statistical Learning via the Alternating Direction Method of Multipliers". In: *Foundations and Trends in Machine Learning* 3.1 (2011), pp. 1–122. DOI: 10.1561/2200000016.

[7] Edgar Buckingham. "On Physically Similar Systems; Illustrations of the Use of Dimensional Equations". In: *Physical Review* 4.4 (1914), pp. 345–376. DOI: 10.1103/PhysRev.4.345.

[8] Apostolos Chalkis and Vissarion Fisikopoulos. *volesti: Volume Approximation and Sampling of Convex Bodies*. R package and C++ library. 2021. URL: https://github.com/GeomScale/volesti.

[9] Philippe G. Ciarlet. *The Finite Element Method for Elliptic Problems*. Classics edition. SIAM, 2002. DOI: 10.1137/1.9780898719208.

[10] Richard L. Dykstra. "An Algorithm for Restricted Least Squares Regression". In: *Journal of the American Statistical Association* 78.384 (1983), pp. 837–842. DOI: 10.1080/01621459.1983.10477029.

[11] Alexandre Girard. "Dimensionless Policies Based on the Buckingham Π Theorem: Is This a Good Way to Generalize Numerical Results?" In: *Mathematics* 12.5 (2024), p. 709. DOI: 10.3390/math12050709.

[12] Jonathan Gorard et al. "Improved Dimensionality Reduction for Inverse Problems in Nuclear Fusion and High-Energy Astrophysics". In: (2025). arXiv:2505.03849. DOI: 10.48550/arXiv.2505.03849.

[13] Kai Hormann and N. Sukumar, eds. *Generalized Barycentric Coordinates in Computer Graphics and Computational Mechanics*. CRC Press, 2017. DOI: 10.1201/9781315153452.

[14] Josip Kir Hromatko et al. "Direct Transfer of Optimized Controllers to Similar Systems Using Dimensionless MPC". In: (2025). arXiv:2512.08667. DOI: 10.48550/arXiv.2512.08667.

[15] Qi Huangfu and J. A. J. Hall. "Parallelizing the Dual Revised Simplex Method". In: *Mathematical Programming Computation* 10.1 (2018). HiGHS solver: https://highs.dev, pp. 119–142. DOI: 10.1007/s12532-017-0130-5.

[16] Mokbel Karam and Tony Saad. "BuckinghamPy: A Python Software for Dimensional Analysis". In: *SoftwareX* 16 (2021), p. 100851. DOI: 10.1016/j.softx.2021.100851.

[17] Ming-Jun Lai and Larry L. Schumaker. *Spline Functions on Triangulations*. Cambridge University Press, 2007. DOI: 10.1017/CBO9780511721588.

[18] Peter McMullen. "The Maximum Numbers of Faces of a Convex Polytope". In: *Mathematika* 17.2 (1970), pp. 179–184. DOI: 10.1112/S0025579300002850.

[19] G. M. Morton. "A Computer Oriented Geodetic Data Base and a New Technique in File Sequencing". In: *IBM Technical Report* (1966).

[20] Yurii Nesterov and Arkadii Nemirovskii. "Multi-Parameter Surfaces of Analytic Centers and Long-Step Surface-Following Interior Point Methods". In: *Mathematics of Operations Research* 23.1 (1998), pp. 1–38. DOI: 10.1287/moor.23.1.1.

[21] Hanna Opdahl and David Jensen. "Dimensional Analysis and Optimization of IsoTruss Structures with Outer Longitudinal Members in Uniaxial Compression". In: *Materials* 14.8 (2021). PMC8074371, p. 2079. DOI: 10.3390/ma14082079.

[22] Miquel Romero-Obon et al. "Scale-Agnostic Models Based on Dimensionless Quality by Design as Pharmaceutical Development Accelerator". In: *Pharmaceuticals* 18.7 (2025), p. 1033. DOI: 10.3390/ph18071033.

[23] Marco Salazar-Meza et al. "Modeling the Ultrasonic Micro-Injection Molding Process Using the Buckingham Pi Theorem". In: *Polymers* 15.18 (2023). PMC10534782, p. 3779. DOI: 10.3390/polym15183779.

[24] Y. Sarazin et al. "Impact of Scaling Laws on Tokamak Reactor Dimensioning". In: *Nuclear Fusion* 60.1 (2020), p. 016010. DOI: 10.1088/1741-4326/ab48a5.

[25] Francesco Scibilia, Sorin Olaru, and Morten Hovd. "Approximate Explicit Linear MPC via Delaunay Tessellation". In: *Proceedings of the European Control Conference (ECC)*. 2009, pp. 2833–2838.

[26] Thomas Strohmer and Roman Vershynin. "A Randomized Kaczmarz Algorithm with Exponential Convergence". In: *Journal of Fourier Analysis and Applications* 15.2 (2009), pp. 262–278. DOI: 10.1007/s00041-008-9030-4.

[27] The CGAL Project. *CGAL User and Reference Manual*. Version 6.x. 2024. URL: https://doc.cgal.org/.

[28] Pål Tøndel, Tor Arne Johansen, and Alberto Bemporad. "Evaluation of Piecewise Affine Control via Binary Search Tree". In: *Automatica* 39.5 (2003), pp. 945–950. DOI: 10.1016/S0005-1098(02)00308-4.